
프로젝트 결과보고서

TCP 다중 스레드 기반 드론 군집 제어



교과명	네트워크프로그래밍(12)
교수명	
학부	
전공	
학년	3학년
학번	
이름	
제출일	2026. 6. 12. (금)

— < 목 차 > —

I. 프로젝트 개요		3
II. 시스템 구조		4
III. 주요 소스 설명		5
IV. 동작 방식		8
V. 동작 설명		9
VI. 안정성 및 동기화		12
VII. 결 론		12
별첨. 실행 방법		12

1 프로젝트 개요

○ 목적

- 기지국 서버 1대가 TCP로 접속한 $N(\geq 3)$ 대의 드론 클라이언트와 통신하며, 각 드론의 위치를 실시간으로 파악하고 이동 명령을 내려 다음 두 단계의 군집 비행을 수행한다.

주요 내용

1차 이동(집결)	모든 드론을 고도 100m 중심, 반경 30m 내에 상호 간격 $\geq 10m$ 로 모은다.
2차 이동(좌측 이동)	1차 완료 후 모든 드론을 왼쪽으로 50m씩 이동시키고 종료한다.

○ 동작 영역 및 좌표계 (가정)

- 명세가 정하지 않은 부분은 다음과 같이 확정하였다.

항 목	값
좌표계	기지국을 원점(0,0). X = 좌우(너비), Y = 고도. 단위 m
X 범위	-100 ~ +100 (기지국 기준 좌우 폭)
Y 범위	20 ~ 200 (지상 20m ~ 최고 200m)
집결 중심	(0, 100)
이동 속도	2 m/tick
보고·제어 주기	200 ms
목표 도달 오차	1.0 m

○ 개발 결과물 구성

파 일	책 임
protocol.h	서버·클라이언트 공유 메시지 구조체 및 상수
server.c	기지국 서버 (accept·드론 수신·중앙 제어·화면 렌더)
drone.c	드론 클라이언트 (위치 보고·명령 수신·이동 시뮬레이션)

2 시스템 구조

○ 구성

- Acceptor(main) —연결당—▶ DroneHandler ×N
- ControlLoop : 페이즈 관리 + MOVE_CMD 유니캐스트
- RenderLoop : 레이더/상태표/메시지로그 실시간 렌더
- 공유상태 g_drones[] / g_phase (g_mutex 보호)

○ 쓰레드 구성

- 예제의 멀티쓰레드 골격(연결당 쓰레드 + 단일 뮤텍스)을 그대로 계승하고, 드론 제어에 필요한 중앙 제어 쓰레드와 시각화 쓰레드를 추가하였다.

구성요소	예 제	본 프로젝트
클라이언트 소켓 보관	clntSocks[] 배열	DroneState g_drones[] (소켓+위치+상태)
동기화	hMutex	g_mutex (동일 방식)
연결당 쓰레드	HandleClnt	DroneHandler (위치 보고 수신)
전체 전송	SendMsg	개별 MOVE_CMD 유니캐스트
—	(없음)	ControlLoop (페이즈 관리) 신규
—	(없음)	RenderLoop (실시간 시각화) 신규

3

주요 소스 설명

○ 공유 헤더 protocol.h — 메시지 정의

```
#define MSG_POSITION_REPORT 1 /* 드론 -> 서버 : 현재 위치 보고 */
#define MSG_MOVE_CMD 2 /* 서버 -> 드론 : 목표 위치로 이동 */
#define MSG_TERMINATE 4 /* 서버 -> 드론 : 종료 */

typedef struct {
    int type; /* 메시지 종류 */
    int droneld; /* 드론 식별자 */
    float x, y; /* 위치/목표 좌표 */
    int seq; /* 순서 번호(로깅용) */
} Message;
```

- 서버와 클라이언트가 동일한 고정 크기 구조체를 공유하므로, 송수신 시 형 변환 없이 sizeof(Message) 단위로 주고받을 수 있다.

○ 서버 — TCP 초기화 및 accept 루프

- 예제와 동일하게 socket → bind → listen → accept 순서로 동작하며, 접속이 들어올 때마다 빈 슬롯에 드론을 등록하고 전용 수신 쓰레드를 생성한다.

```
g_servSock = socket(PF_INET, SOCK_STREAM, 0);
bind(g_servSock, (SOCKADDR*)&servAdr, sizeof(servAdr));
listen(g_servSock, 5);

while (g_running) {
    SOCKET hCln = accept(g_servSock, (SOCKADDR*)&clntAdr, &clntAdrSz);
    if (hCln == INVALID_SOCKET) break; /* 종료 시 소켓 닫힘 */

    WaitForSingleObject(g_mutex, INFINITE);
    int slot = RegisterDrone(hCln); /* 빈 슬롯 등록, id 부여 */
    ReleaseMutex(g_mutex);

    _beginthreadex(NULL, 0, DroneHandler, (void*)(intptr_t)slot, 0, NULL);
}
```

○ 서버 — 드론별 수신 쓰레드 DroneHandler

- 각 드론의 POSITION_REPORT를 받아 공유 상태 테이블을 갱신한다.
- 공유 자원 접근은 모두 뮤텁스로 보호한다.

```
unsigned WINAPI DroneHandler(void *arg) {
    int slot = (int)(intptr_t)arg;
    SOCKET sock = g_drones[slot].sock;
    Message m;
    while (g_running) {
        int r = RecvFixed(sock, (char*)&m, sizeof(m));
        if (r == 0 || r == SOCKET_ERROR) break; /* 연결 종료/오류 */
        if (m.type == MSG_POSITION_REPORT) {
            WaitForSingleObject(g_mutex, INFINITE);
            g_drones[slot].x = m.x; g_drones[slot].y = m.y;
            g_drones[slot].reported = 1;
            ReleaseMutex(g_mutex);
            AddLog(...); /* 메시지 로그 패널에 RX 기록 */
        }
    }
    /* 연결 정리: 슬롯 비활성화 + 소켓 닫기 */
}
```

○ 서버 — 중앙 제어 쓰레드 ControlLoop (핵심)

- 시스템을 페이즈 상태기계로 운영한다.

```
PHASE_INIT → 드론 ≥3 접속 시 1차 집결 목표 산출·송신 → PHASE_GATHERING
PHASE_GATHERING → 전원 반경30m·간격10m 충족 시 → PHASE1_DONE
PHASE1_DONE → 2차 목표(각자 X-50) 산출·송신 → PHASE_SHIFTING
PHASE_SHIFTING → 전원 2차 목표 도달 시 TERMINATE 송신 → PHASE_DONE
```

- 임계구역 안에서는 판단과 목표 산출만 하고, 실제 send()는 잠금을 풀 뒤 수행하여 교착을 방지한다.

```
WaitForSingleObject(g_mutex, INFINITE);
/* ... 페이즈별 판정 + 보낼 명령 목록(out[])을 복사 ... */
g_phase = nextPhase;
ReleaseMutex(g_mutex);

for (i = 0; i < outN; i++) /* 잠금 밖에서 송신 */
    SendFixed(out[i].sock, (const char*)&out[i].msg, sizeof(Message));
```

- 1차 집결 목표는 중심 (0,100) 둘레 반경 15m 원주에 드론을 균등 배치한다.

```
float ang = 2*PI*k / n;
tx = CENTER_X + RING_RADIUS * cos(ang); /* RING_RADIUS = 15 */
ty = CENTER_Y + RING_RADIUS * sin(ang);
```

→ 반경 15m ≤ 30m 를 만족하고, N=3일 때 드론 간 간격 ≈ 26m ≥ 10m 를 자연히 만족한다.

○ 클라이언트 — 위치 보고 쓰레드 ReportThread

- 주기마다 목표를 향해 한 틱 이동(선형 보간)한 뒤 위치를 보고한다.
- 1차 명령을 받기 전에는 영역 내 임의 목표를 향해 배회한다.

```
if (!g_commanded && 목표도달) { /* 명령 전: 새 임의 목표 */
    g_tx = frand(WORLD_X_MIN, WORLD_X_MAX);
    g_ty = frand(WORLD_Y_MIN, WORLD_Y_MAX);
}
float dist = sqrt(dx*dx + dy*dy); /* 점진 이동 */
if (dist <= MOVE_STEP) { g_x = g_tx; g_y = g_ty; }
else { g_x += MOVE_STEP*dx/dist; g_y += MOVE_STEP*dy/dist; }
clamp(g_x, -100, 100); clamp(g_y, 20, 200);

m.type = MSG_POSITION_REPORT; m.x = g_x; m.y = g_y;
SendFixed(sock, (char*)&m, sizeof(m));
```

○ 클라이언트 — 명령 수신 쓰레드 CommandThread

```
int r = RecvFixed(sock, (char*)&m, sizeof(m));
if (m.type == MSG_MOVE_CMD) { /* 목표 갱신 */
    g_tx = m.x; g_ty = m.y; g_commanded = 1;
} else if (m.type == MSG_TERMINATE) { /* 종료 */
    g_alive = 0; break;
}
```

○ 신뢰성 있는 송수신 래퍼 (공통)

- TCP는 바이트 스트림이라 한 번의 recv()가 메시지의 일부만 줄 수 있다.
- 예제와 달리 본 구현은 고정 크기 구조체를 쓰므로, 정확히 sizeof(Message)만큼 채울 때까지 반복한다.

```
int RecvFixed(SOCKET s, char *buf, int len) {
    int got = 0, r;
    while (got < len) {
        r = recv(s, buf+got, len-got, 0);
        if (r == 0 || r == SOCKET_ERROR) return r;
        got += r;
    }
    return got;
} /* SendFixed 도 동일하게 전량 전송을 보장 */
```

4 메시지 구조 및 동작 방식

○ 메시지 구조

- 모든 메시지는 동일한 Message 구조체(고정 크기)이며, type 필드로 종류를 구분한다.

필드	타입	의미
type	int	메시지 종류 (아래 표)
droneld	int	드론 식별자
x, y	float	위치(보고) 또는 목표 좌표(명령)
seq	int	순서 번호 (로깅·추적용)

○ 메시지 종류별 동작

메시지	방향	내용	수신 측 동작
POSITION_REPORT	드론 → 서버	현재 위치 (x,y)	서버가 해당 드론의 상태 테이블 위치를 갱신, 로그 기록
MOVE_CMD	드론 ← 서버	목표 위치 (x,y)	드론이 목표를 갱신하고 그 방향으로 점진 이동 시작
TERMINATE	드론 ← 서버	종료 신호	드론이 소켓을 닫고 프로그램 종료

5

실행 화면 중심 동작 설명



○ 서버 콘솔 구성

- 서버 화면은 레이더 / 상태표 / 메시지 로그 3개 패널로 구성되며 10 FPS로 갱신된다.
- SetConsoleCursorPosition로 커서를 좌상단으로 되돌려 화면 깜빡임 없이 다시 그린다.

○ INIT — 임의 배치/배회

- 드론 3대가 접속하여 영역 내 임의 위치에서 움직인다.
- 상태는 CONN, 페이지는 INIT.
- 메시지 로그에 각 드론의 RX POS(위치 보고)가 연속 기록된다.



○ GATHERING — 1차 집결 진행

- 서버가 각 드론에 집결 목표를 담은 MOVE_CMD를 전송(TX MOVE)한 직후의 화면.
- 드론들이 레이더 상에서 중심 +(0,100) 쪽으로 모여든다.
- 상태표의 중심거리가 점차 줄어드는 것을 확인할 수 있다.

○ GATHERED — 1차 완료

- 모든 드론의 중심거리 $\leq 30m$, 최소간격 $\geq 10m$ 가 충족되어 집결중심 반경내: 3/3으로 표시.
- 상태가 GTHD로 바뀌고 페이지가 GATHERED로 전이된다. → 제약 충족의 직접 증빙.

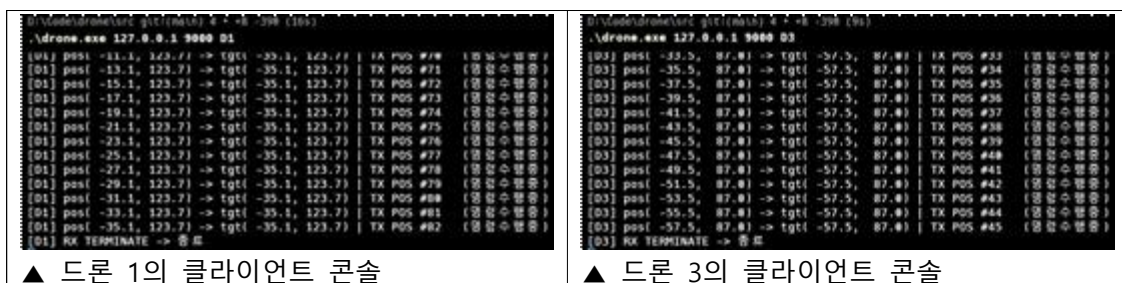


○ SHIFTING — 2차 좌측 이동

- 서버가 전원에게 현재 X - 50 목표의 MOVE_CMD를 전송.
- 레이더에서 군집 전체가 좌측으로 평행 이동한다.
- 목표X 값이 기존보다 50 작아진 것을 상태표에서 확인할 수 있다.

○ DONE — 종료

- 전원이 2차 목표에 도달하면 서버가 TERMINATE(TX TERM)를 전송하고 모든 프로그램이 종료된다.



○ 드론 클라이언트 콘솔

- 각 드론은 위치 보고와 명령 수신을 한 줄씩 로그로 출력한다.

6

안정성 및 동기화

- 실행 안정성(평가 항목)을 위해 다음을 적용하였다.

주요 내용

공유 자원 보호	g_drones[]·g_phase의 모든 접근을 g_mutex로 보호
교착 방지	임계구역 내에서 블로킹 send() 금지 — 잠금 밖에서 전송
부분 송수신	RecvFixed/SendFixed로 메시지 전량 송수신 보장
연결 종료 처리	recv() 0/오류 시 슬롯 비활성화 + 소켓 정리
출력 경합	콘솔 출력은 RenderLoop 단일 스레드로 일원화
자원 해제	종료 시 소켓·뮤텝스 정리 후 WSACleanup()

7

결 론

- 수업에서 배포한 멀티스레드 채팅 예제의 골격을 기반으로, TCP 다중 스레드 서버가 3대 이상의 드론을 동시에 제어하는 군집 제어 시스템을 구현하였다.
- 고정 크기 메시지 프로토콜과 페이지 상태기계를 설계하여 1차 집결(반경 30m·간격 10m)과 2차 좌측 이동(50m)을 단계적으로 수행하고, 레이더·상태표·메시지 로그 3패널 시각화를 통해 드론의 움직임과 메시지 송수신 상황을 한눈에 확인할 수 있도록 하였다.
- 모든 공유 자원 접근을 뮤텝스로 보호하고 메시지 프레임링을 처리하여 실행 안정성을 확보하였다.

별 첨

실행 방법

```
실행 (서버 1 + 드론 3)
, Wserver.exe 9000
, Wdrone.exe 127.0.0.1 9000 D1
, Wdrone.exe 127.0.0.1 9000 D2
, Wdrone.exe 127.0.0.1 9000 D3
```